AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

Aplikacja do trenowania

Autor:
Michał Tokarz
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań – Wizja Produktu **3**

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX) 3
1.2. Wizualizacja postępów i Dashboard 4
1.3. Niewidzialna technologia – Sensory jako benefity 4
1.4. Aspekt społecznościowy i grywalizacja 6
1.5. Proces wdrożenia (Onboarding) 6

2. Określenie wymagań szczegółowych – Architektura i Implementacja **7**

2.1. Stos Technologiczny i Środowisko Programistyczne 7
2.2. Architektura Systemu i Wzorzec MVVM 8
2.3. Integracja z natywnym API Androida (Sensory) 8
2.4. Warstwa Persystencji – Baza Danych Firebird 9

3. Projektowanie **10**

3.1. Przygotowanie narzędzi i system kontroli wersji Git 10
3.2. Wybór bibliotek i architektura klas 10
3.3. Szkic layoutów i zasady nawigacji (Flow aplikacji) 11
3.4. Algorytmy i pseudo-kody 12

4. Implementacja **13**

5. Testowanie **14**

6. Podręcznik użytkownika **15**

Literatura **16**

Spis rysunków **16**

Spis tabel **17**

Spis listingów **18**

1. Ogólne określenie wymagań – Wizja Produktu

W dobie powszechnej cyfryzacji i rosnącej świadomości na temat zdrowego stylu życia, na rynku aplikacji fitness panuje ogromna konkurencja. Zdecydowana większość dostępnych rozwiązań to jednak narzędzia przeładowane niepotrzebnymi funkcjami, skomplikowane w obsłudze i zniechęcające swoim interfejsem. Przedmiotem niniejszej dokumentacji jest innowacyjna aplikacja mobilna dedykowana na urządzenia z systemem Android, która zrywa z tym schematem.

Nasz produkt został zaprojektowany z myślą o jednym, nadrzędnym celu: dostarczeniu użytkownikowi absolutnie bezproblemowego, intuicyjnego i wizualnie oszałamiającego doświadczenia, które motywuje do codziennej aktywności fizycznej. Zamiast zmuszać użytkownika do dostosowywania się do technologii, sprawiliśmy, że to technologia – ukryta pod płaszczykiem pięknego interfejsu – dostosowuje się do niego.

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX)

Wygląd aplikacji to pierwszy i najważniejszy punkt styku klienta z naszym produktem. Dlatego warstwa wizualna została zaprojektowana w oparciu o najnowsze trendy *Material Design* ze szczególnym naciskiem na minimalizm, czytelność oraz wysoki kontrast.

Kolorystyka i typografia

Zrezygnowaliśmy z krzykliwych, rozpraszających barw na rzecz eleganckiej, głębokiej czerni (idealnie współgrającej z ekranami AMOLED, co dodatkowo oszczędza baterię smartfona) oraz energetyzujących akcentów w odcieniu trawiastej, nasyconej zieleni. Ten wybór kolorystyczny nie jest przypadkowy – czerń symbolizuje profesjonalizm i skupienie, podczas gdy zieleń to uniwersalny symbol zdrowia, natury, witalności oraz pomyślnie zrealizowanego celu (tzw. *green light*). Zastosowana typografia opiera się na bezszeryfowych, grubych fontach (typu *display*), które gwarantują, że nawet w pełnym biegu, w ostrym słońcu, użytkownik bez problemu odczyta swoje tętno, czas czy przebyty dystans.

Nawigacja oparta na gestach

Filozofia interfejsu opiera się na zasadzie: „mniej klikania, więcej biegania". Wyeliminowaliśmy skomplikowane menu na rzecz nawigacji opartej na naturalnych gestach. Przesunięcie palcem w lewo ukazuje panel z muzyką, w prawo – szczegółową mapę, a pociągnięcie w dół rozwija statystyki bieżącej sesji. Główny przycisk akcji (tzw. *Floating Action Button*) to potężny, pulsujący na zielono element, który wręcz zaprasza

do rozpoczęcia treningu.

1.2. Wizualizacja postępów i Dashboard

Gromadzona przez system ogromna ilość danych (zapisywana w wydajnej bazie Firebird) jest bezużyteczna, jeśli nie zostanie odpowiednio zaprezentowana. Nasza aplikacja to w dużej mierze zaawansowane narzędzie do wizualizacji danych.

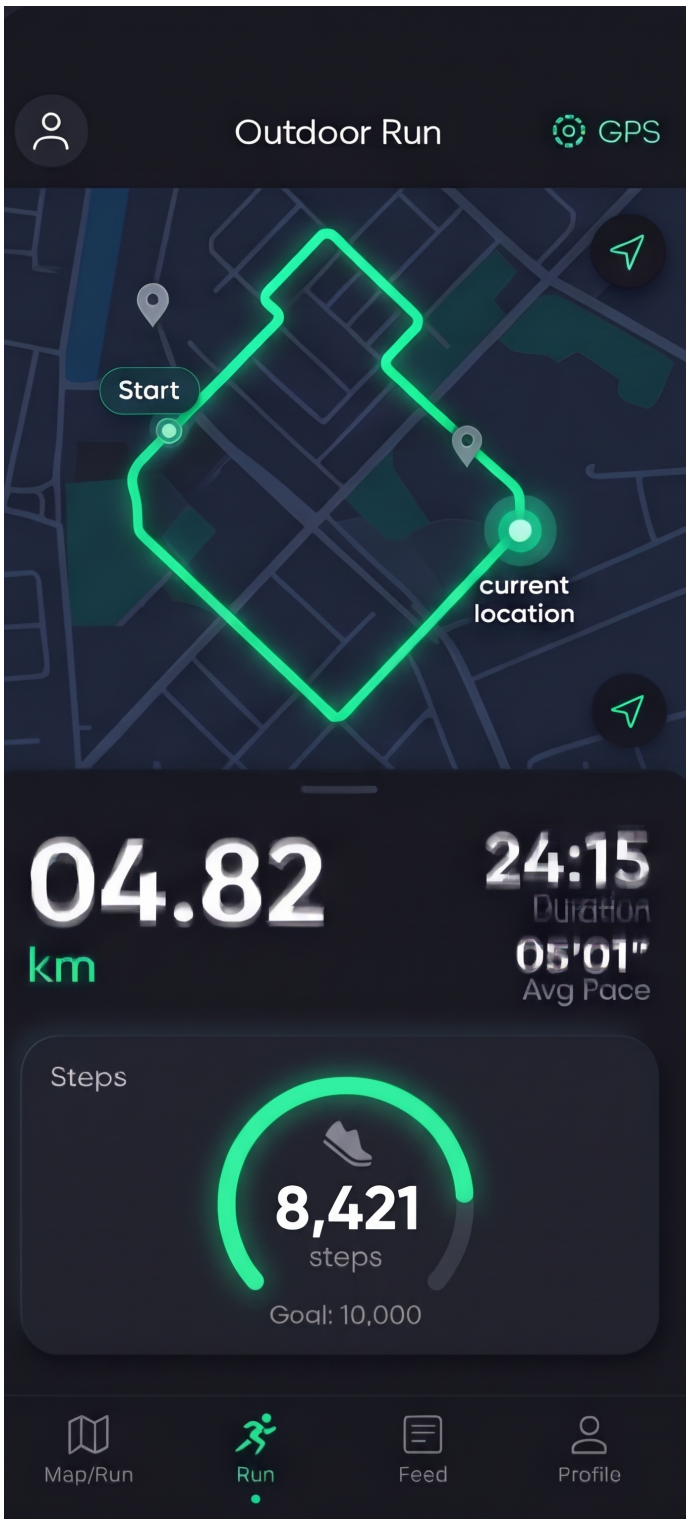
- **Pierścienie aktywności:** Inspirowane najlepszymi rozwiązaniami na rynku, wprowadziliśmy koncepcję „zamykania pierścieni". Użytkownik widzi na ekranie głównym trzy nachodzące na siebie, neonowe okręgi symbolizujące dystans, spalone kalorie i czas aktywności. Cel na każdy dzień to ich domknięcie.
- **Interaktywne mapy i heat-mapy (Ślad GPS):** Przebyta trasa nie jest zwykłą, nudną linią na mapie. Zastosowaliśmy płynne gradienty, które zmieniają kolor w zależności od tempa użytkownika – odcinki, gdzie tempo było najwyższe, żarzą się na czerwono, a miejsca spacerowe przybierają chłodne odcienie błękitu.
- **Płynne wykresy:** Analiza powysiłkowa prezentowana jest za pomocą wygładzonych wykresów krzywoliniowych, które użytkownik może płynnie skalować (szczypiąc ekran) i przesuwać. Każdy punkt na wykresie reaguje na dotyk, wyświetlając w dymku precyzyjną wartość w danym ułamku sekundy.

1.3. Niewidzialna technologia – Sensory jako benefity

Od strony technicznej, aplikacja jest majstersztykiem wykorzystującym GPS, krokomierz, żyroskop i moduły biometryczne. Jednak dla użytkownika końcowego są to wyłącznie realne korzyści ubrane w magiczne funkcje.

Twoje ciało to Twoje hasło (Biometria)
Zrezygnowaliśmy z tradycyjnych ekranów logowania. Kto chciałby przypominać sobie hasło, stojąc w deszczu i przygotowując się do biegu? Dzięki integracji z systemowym czytnikiem linii papilarnych i skanerem twarzy, aplikacja wita użytkownika natychmiastowo. Na ekranie pojawia się jedynie elegancka animacja skanowania, a po ułamku sekundy interfejs płynnie przechodzi do panelu startowego.

Inteligentny detektor ruchu (Żyroskop i Krokomierz)
Wyobraźmy sobie sytuację: biegacz zatrzymuje się na światłach. Zamiast nerwowo wyciągać telefon i pauzować aplikację, system robi to sam. Wykorzystując zaawansowane algorytmy analizujące dane z żyroskopu i krokomierza, aplikacja wyczuwa, że



Rys. 1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji – minimalistyczny ekran trwającego treningu z wykorzystaniem ciemnego motywu i zielonych akcentów.

ruch ustał, i z subtelnym wibrującym sygnałem informuje o automatycznej pauzie. Ekran delikatnie przygasa, a gdy tylko użytkownik ruszy, licznik płynnie wznawia pracę.

Zawsze na właściwej ścieżce (GPS)
Wykorzystanie modułu geolokalizacji zostało zoptymalizowane tak, aby działać z najwyższą dokładnością bez drenowania baterii. Użytkownik na bieżąco widzi swoją pozycję podążającą za nim na przepięknie wyrenderowanej mapie wektorowej, która reaguje na obrót telefonu (dzięki żyroskopowi), zachowując perspektywę 3D.

1.4. Aspekt społecznościowy i grywalizacja

W dzisiejszych czasach trening, którym nie można się pochwalić, to dla wielu użytkowników trening stracony. Zrozumieliśmy to, projektując zintegrowany moduł społecznościowy.

Zamiast suchych tabel z wynikami, aplikacja po każdym treningu generuje przepiękne, gotowe do udostępnienia w mediach społecznościowych (Instagram, Facebook) „karty treningowe". Użytkownik może nałożyć swój ślad GPS i nałożone na niego, estetyczne statystyki bezpośrednio na zdjęcie zrobione na trasie. Dodaliśmy system wirtualnych odznak – za przebiegnięcie pierwszych 10 kilometrów, za trening w deszczu, czy za zamknięcie tygodnia z kompletem pierścieni aktywności. Zdobyciu każdej odznaki towarzyszy satysfakcjonująca animacja 3D i wibracje haptyczne telefonu.

1.5. Proces wdrożenia (Onboarding)

Zamiast standardowej procedury instalacji, proces wejścia w świat naszej aplikacji to tzw. *onboarding* – interaktywny samouczek. Po pobraniu aplikacji ze sklepu, użytkownik witany jest pełnoekranowym materiałem wideo w tle, na którym widać biegnącego sportowca. Krok po kroku, w formie przyjaznych pytań („Jak masz na imię?", „Jaki jest Twój cel?"), aplikacja zbiera dane początkowe. Wszystko to okraszone jest miękkimi przejściami (tzw. *fade-in* oraz *slide-up*), co buduje ogromne poczucie produktu w wersji Premium już od pierwszych sekund używania.

Podsumowując, nasz produkt to nie tylko zbiór linijek kodu i odpytywań sprzętowych sensorów. To starannie zaprojektowane narzędzie motywacyjne, w którym design, typografia i użyteczność stoją na równi z inżynierią oprogramowania.

Tab. 1.1. Przełożenie technologii na korzyści wizualne i UX

Wykorzystany moduł	**Funkcja w aplikacji**	**Korzyść wizualna / UX dla użytkownika**
Skaner Biometryczny	Szybkie logowanie	Ekran logowania znika w 0.5s, płynna animacja odblokowania kłódki, brak frustracji.
Odbiornik GPS	Śledzenie trasy	Gradientowa linia trasy na mapie 3D (tzw. heat-mapa tempa), poczucie pełnej kontroli.
Żyroskop i Krokomierz	Auto-Pauza	Ekran inteligentnie przyciemnia się na postoju i rozbłyska przy starcie; oszczędność baterii.
Baza Danych (Firebird)	Zapis historii	Błyskawiczne wczytywanie wieloletniej historii na wygładzonych wykresach, bez irytującego ładowania.

2. Określenie wymagań szczegółowych – Architektura i Implementacja

Niniejszy rozdział stanowi kompletną specyfikację techniczną projektowanego systemu z perspektywy programistycznej. Dokument definiuje stos technologiczny, wymagane wzorce projektowe, koncepcję struktur bazodanowych oraz rygorystyczne zasady zarządzania pamięcią i wątkami w środowisku uruchomieniowym .NET 10 na platformie Android.

2.1. Stos Technologiczny i Środowisko Programistyczne

Aplikacja rozwijana jest w oparciu o nowoczesny stos technologiczny firmy Microsoft, zoptymalizowany pod kątem wieloplatformowego wytwarzania oprogramowania przy użyciu jednego kodu źródłowego (ang. *Single Codebase*).

Kompilacja i środowisko:
Projekt bazuje na języku **C# w wersji 12** oraz platformie **.NET SDK 10**. Zamiast polegać na ciężkich zintegrowanych środowiskach (IDE), proces deweloperski opiera się na lekkim edytorze Visual Studio Code. Od strony programisty wymaga to płynnego posługiwania się interfejsem wiersza poleceń (CLI). Programista jest

odpowiedzialny za ręczne zarządzanie cyklem życia projektu za pomocą narzędzia systemowego `dotnet`. Do jego zadań należy inicjalizacja pobierania zależności (w tym sterownika FirebirdClient) oraz parametryzacja procesu budowania tak, aby kod C# został skompilowany bezpośrednio do natywnych bibliotek współdzielonych Androida (`.so`) z wykorzystaniem kompilacji Ahead-of-Time (AOT), co drastycznie zminimalizuje czas zimnego startu aplikacji.

2.2. Architektura Systemu i Wzorzec MVVM

Kod źródłowy musi zostać zorganizowany ściśle według paradygmatu **MVVM (Model-View-ViewModel)**, wspieranego przez kontener wstrzykiwania zależności (*Dependency Injection Container*).

Programista ma za zadanie rozdzielić logikę na trzy niezależne warstwy:

- **Model:** Warstwa dostępu do danych, w której zaimplementowane zostaną encje biznesowe oraz repozytoria komunikujące się z bazą danych Firebird.
- **ViewModel:** Silnik logiczny aplikacji. To tutaj programista zaimplementuje interfejs `INotifyPropertyChanged`, powiąże komendy z interfejsem użytkownika oraz zaimplementuje algorytmy obliczeniowe (np. wzór haversine'a do przeliczania dystansu GPS).
- **View:** Całkowicie pasywna warstwa prezentacji, która wyłącznie subskrybuje zmiany właściwości ViewModelu poprzez mechanizm *Data Bindingu*.

Kluczowym obowiązkiem programisty jest odciążenie głównego wątku interfejsu użytkownika (UI Thread). Wszelkie operacje wejścia/wyjścia oraz ciężkie filtry matematyczne dla żyroskopu muszą być realizowane z rygorystycznym wykorzystaniem biblioteki TPL (*Task Parallel Library*) oraz asynchroniczności bazującej na słowach kluczowych `async` / `await`.

2.3. Integracja z natywnym API Androida (Sensory)

Silnik aplikacji komunikuje się z warstwą sprzętową za pośrednictwem dedykowanych bindingów C#. Aby system operacyjny Android nie "zabił"procesu aplikacji podczas długiego treningu przy zablokowanym ekranie, programista musi zaimplementować usługę działającą w tle typu **Foreground Service** powiązaną z trwałą notyfikacją systemową.

Zarządzanie uprawnieniami:
Programista jest zobligowany do odpowiedniej konfiguracji pliku manifestu aplikacji. Należy w nim statycznie zadeklarować dostęp do precyzyjnej lokalizacji (również w tle), detekcji aktywności fizycznej, sprzętowego krokomierza, żyroskopu oraz modułu biometrycznego. Oprócz wpisów w manifeście, kod aplikacji musi obsługiwać tzw. *Runtime Permissions* – dynamiczne monity proszące użytkownika o zgodę na użycie sensorów w momencie pierwszego uruchomienia treningu.

Moduł Geotrackingu (GPS) i Fuzja Czujników:
Z poziomu kodu należy skonfigurować natywnego menedżera lokalizacji, precyzyjnie dobierając parametry odświeżania (np. minimalny czas i dystans między odczytami), aby zminimalizować efekt "pływania"sygnału. Dodatkowo programista musi napisać algorytm korelujący dane z różnych czujników. Przykładowo, funkcja automatycznej pauzy ma analizować jednoczesny przyrost kroków oraz wariancję prędkości kątowej z żyroskopu, klasyfikując w ten sposób, czy użytkownik faktycznie biegnie, czy jedzie pojazdem.

2.4. Warstwa Persystencji – Baza Danych Firebird

Decyzja o wykorzystaniu silnika relacyjnej bazy danych **Firebird** podyktowana jest jego wydajnością. Programista musi zrezygnować z ociężałych frameworków ORM (jak Entity Framework) na rzecz czystego ADO.NET lub lekkich maperów, aby zapewnić najwyższą przepustowość podczas masowego wstawiania tysięcy punktów GPS (tzw. *Bulk Insert*).

Projekt struktury danych:
Zadaniem zespołu deweloperskiego jest zaprojektowanie bazy w trzeciej postaci normalnej (3NF). Wymagane będzie utworzenie tabel:

- **Tabeli użytkowników**, przechowującej unikalne identyfikatory, parametry fizyczne oraz kryptograficzne skróty haseł (np. SHA-256 z wbudowaną solą).
- **Tabeli nagłówkowej sesji treningowych**, służącej do agregacji podstawowych danych o aktywności (czas trwania, całkowity dystans, spalone kalorie).
- **Tabeli punktów trasowych (Trackpoints)**, dedykowanej do zapisu setek tysięcy współrzędnych GPS z precyzyjnymi znacznikami czasu.

Od strony optymalizacyjnej, programista ma obowiązek założyć indeksy B-drzewa na kluczach obcych i kolumnach chronologicznych, aby przyspieszyć ładowanie historii treningów.

Bezpieczeństwo i niezawodność:
Wszelkie zapytania wysyłane do serwera Firebird muszą być bezwzględnie parametryzowane w kodzie C# w celu ochrony przed atakami typu *SQL Injection*. Ponadto, operacja zapisu odbytego treningu musi zostać ujęta w jedną, spójną transakcję. Wymaga się również zaimplementowania wzorca *Store-and-Forward* – w przypadku zerwania połączenia z bazą, programista musi zapewnić tymczasowe buforowanie wyników w lokalnej pamięci SQLite do momentu odzyskania dostępu do sieci.

3. Projektowanie

Niniejszy rozdział poświęcony jest fazie projektowania aplikacji. Obejmuje on przygotowanie środowiska programistycznego, wybór kluczowych bibliotek, architekturę klas, projekt interfejsu użytkownika (UI) oraz opis najważniejszych algorytmów wykorzystywanych do przetwarzania danych treningowych.

3.1. Przygotowanie narzędzi i system kontroli wersji Git

Do realizacji projektu wykorzystano środowisko **Visual Studio 2022** (oraz pomocniczo Android Studio dla obsługi i konfiguracji emulatorów). Wymagane było zainstalowanie pakietu SDK dla **.NET 8.0** oraz paczki roboczej narzędzi **.NET MAUI** (Multi-platform App UI), która pozwala na kompilację jednego kodu C# na systemy Android i iOS.

Kontrolę wersji zrealizowano przy pomocy systemu **Git**. Inicjalizacja repozytorium odbyła się lokalnie, po czym utworzono gałąź (branch) o nazwie `Dokumentacja`. Zmiany były systematycznie commitowane, a by uniknąć konfliktów i zbędnego obciążenia repozytorium, dodano odpowiednie filtry (plik `.gitignore`) odrzucające pliki binarne (foldery `bin/` i `obj/`). Kod został ostatecznie zsynchronizowany z platformą GitHub poleceniem `git push`.

3.2. Wybór bibliotek i architektura klas

Projekt bazuje na wbudowanych bibliotekach frameworka .NET MAUI, wykorzystując wzorzec architektoniczny **MVVM** (Model-View-ViewModel), który wymusza separację logiki biznesowej od interfejsu graficznego. Główny podział klas prezentuje tabela 3.1.

Tab. 3.1. Wykaz głównych klas systemowych

Nazwa klasy	Opis i odpowiedzialność
`Exercise` (Model)	Przechowuje dane o pojedynczej aktywności. Zawiera atrybuty dla ćwiczeń siłowych (Sets, Reps, WeightKg) oraz dla aktywności kardio/biegania (DistanceKm, Duration, flaga IsCardio).
`Workout` (Model)	Stanowi agregat obiektów `Exercise`. Przechowuje właściwości `Title` (nazwa treningu) oraz `Date` (data wykonania).
`MainViewModel`	Stanowi warstwę pośrednią. Wykorzystuje interfejs `INotifyPropertyChanged` oraz kolekcję `ObservableCollection<Workout>`, co gwarantuje automatyczne odświeżanie widoku po modyfikacji danych.

3.3. Szkic layoutów i zasady nawigacji (Flow aplikacji)

Interfejs użytkownika (UI) napisano w języku znaczników XAML. Aplikacja opiera się na komponencie `NavigationPage`, który zarządza stosem ekranów. Dokładne założenia działania aplikacji wyglądają następująco:

1. **Ekran główny (MainPage):** Wyświetla przewijaną listę historii treningów (kafelki z nazwą i datą). Na dole znajdują się dwa główne przyciski w układzie kolumnowym (Grid): "Szybki Bieg"oraz "Trening Siłowy".

2. **Ekran Biegu (RunningPage):** Otwiera się po kliknięciu "Szybki Bieg". Górną część ekranu zajmuje kontener (Frame) zarezerwowany pod moduł GPS/Mapy. Dolna sekcja to panel z dużymi wskaźnikami (Dystans, Czas, Tempo) oraz przyciskiem "ROZPOCZNIJ BIEG". Kliknięcie przycisku zmienia jego kolor na czerwony, tekst na "ZAKOŃCZ BIEG"i w tle inicjuje zliczanie czasu.

3. **Ekran Nowego Treningu (AddWorkoutPage):** Wywoływany po wybraniu "Trening Siłowy". Zawiera pola tekstowe do wprowadzania nazwy treningu (Entry) i daty (DatePicker). Znajduje się tu przycisk zapisu, którego wciśnięcie cofnie użytkownika do ekranu głównego (metoda `Navigation.PopAsync()`).

4. **Baza Ćwiczeń (ExercisesDatabasePage):** Ekran zawierający pasek wyszukiwania i pogrupowane przyciski z nazwami ćwiczeń (np. Klatka, Plecy).

Kliknięcie danego ćwiczenia docelowo dodaje je do aktualnie komponowanego treningu siłowego.

3.4. Algorytmy i pseudo-kody

W celu prezentacji prawidłowego działania historii treningów na ekranie głównym, zaistniała potrzeba sortowania dodawanych elementów po dacie (od najnowszego do najstarszego). Wykorzystano podejście oparte na algorytmie sortowania przez wstawianie lub wbudowanych metodach sortujących zapytań LINQ.

Poniżej przedstawiono specyfikację logiczną kluczowych procesów w formie pseudokodów.

Pseudo kod logiki ekranu biegania (stoper i dystans)

```
Zmienna czyBiegnie = Falsz

Funkcja PrzyciskRozpocznijBieg_Klikniecie():
    Jeżeli czyBiegnie == Falsz to:
        czyBiegnie = Prawda
        ZmieńTekstPrzycisku("ZAKOŃCZ BIEG")
        PobierzPoczątkowąLokalizacjęGPS()
        UruchomZegar(co 1 sekundę aktualizuj Czas na ekranie)
    W przeciwnym wypadku:
        czyBiegnie = Falsz
        ZatrzymajZegar()
        ObliczCałkowityDystans()
        ZapiszBiegDoBazyDanych()
        WróćDoEkranuGłównego()
```

Pseudo kod algorytmu dodawania i sortowania historii treningów

```
Funkcja DodajNowyTrening(nowyTrening):
    Dodaj nowyTrening do listy Workouts
    Dla każdego elementu (i) od 0 do wielkości listy Workouts:
        Dla każdego elementu (j) od i+1 do wielkości listy Workouts:
            Jeśli Workouts[i].Date < Workouts[j].Date to:
                Zamień miejscami Workouts[i] oraz Workouts[j]
    Odśwież widok użytkownika (OnPropertyChanged)
```

4. Implementacja

5. Testowanie

6. Podręcznik użytkownika

Spis rysunków

1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji – minimalistyczny ekran trwającego treningu z wykorzystaniem ciemnego motywu i zielonych akcentów. 5

Spis tabel

1.1. Przełożenie technologii na korzyści wizualne i UX 7

3.1. Wykaz głównych klas systemowych 11

Spis listingów